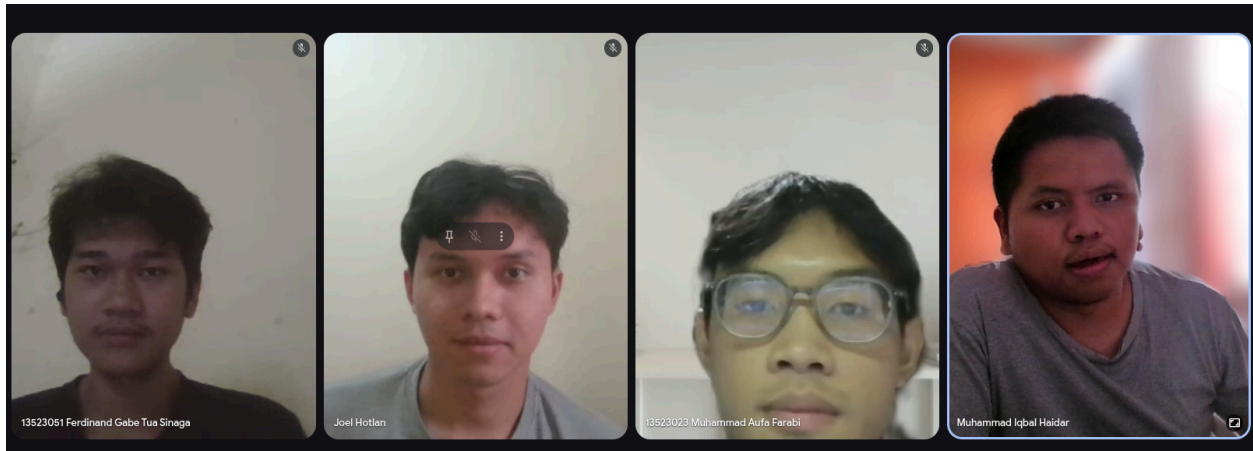


IF2224 TEORI BAHASA FORMAL DAN OTOMATA SEMESTER II TAHUN 2025/2026

PASCAL-S COMPILER



Milestone 1 - Lexical Analysis

Kelompok LPH

Muhammad Aufa Farabi	13523023
Joel Hotlan Haris Siahaan	13523025
Ferdinand Gabe Tua Sinaga	13523051
Muhammad Iqbal Haidar	13523111

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2025**

DAFTAR ISI

DAFTAR ISI.....	2
LANDASAN TEORI.....	3
PERANCANGAN & IMPLEMENTASI.....	3
Diagram.....	3
File Aturan DFA (.json).....	8
Implementasi Lexical Analyzer.....	10
PENGUJIAN.....	11
Kasus 1.....	11
Kasus 2.....	11
Kasus 3.....	12
Kasus 4.....	14
Kasus 5.....	16
KESIMPULAN DAN SARAN.....	17
LAMPIRAN.....	17
REFERENSI.....	18

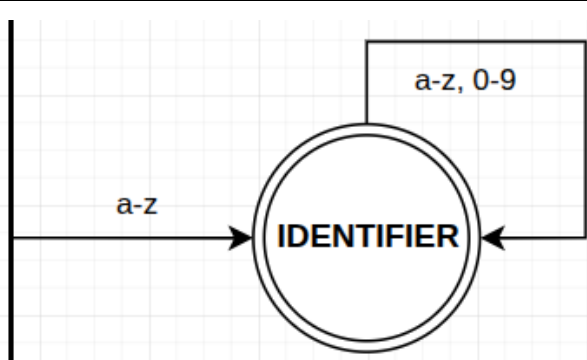
LANDASAN TEORI

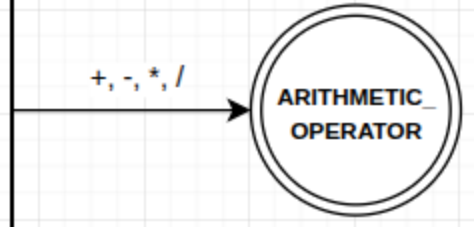
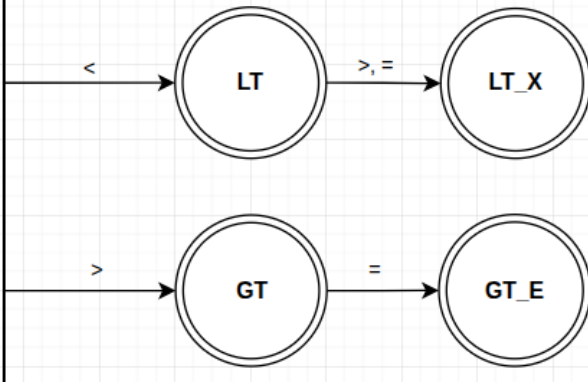
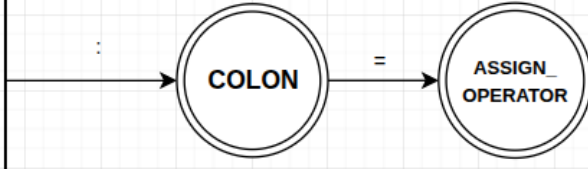
Bahasa pemrograman merupakan sarana untuk melakukan operasi komputasi dalam dunia Informatika. Bahasa ini memungkinkan manusia untuk menuliskan instruksi yang dapat dipahami dan dijalankan komputer. Namun proses mengeksekusi instruksi tidak sederhana. Compiler atau interpreter harus menjalankan serangkaian tahap supaya kode sumber dapat diubah menjadi bentuk yang dapat dieksekusi mesin.

Secara umum, proses kompilasi terdiri dari beberapa tahap yaitu analisis leksikal, analisis sintaks, analisis semantik, optimasi, dan generasi kode. Pada tahap analisis leksikal, komponen compiler yaitu lexer membaca kode sumber yang berupa rangkaian karakter dan mengubahnya menjadi rangkaian token yang merupakan unit terkecil dalam sebuah program. Lexer juga berfungsi untuk mengabaikan bagian-bagian yang tidak relevan seperti spasi, serta mendeteksi kesalahan leksikal, misalnya simbol yang tidak dikenal atau penulisan token yang tidak valid. Output proses ini berupa token yang umumnya berisi dua informasi penting yaitu jenis token dan nilai token.

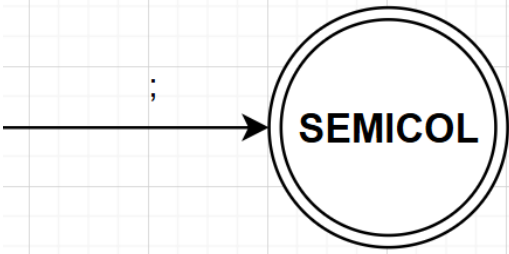
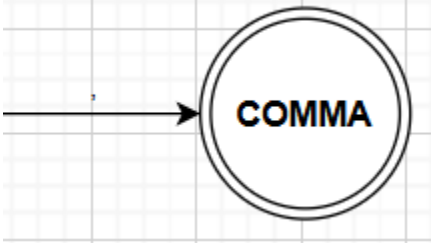
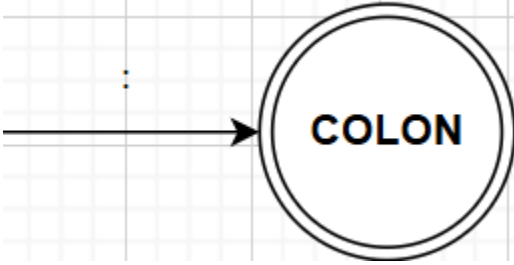
PERANCANGAN & IMPLEMENTASI

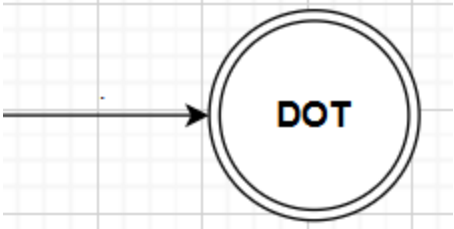
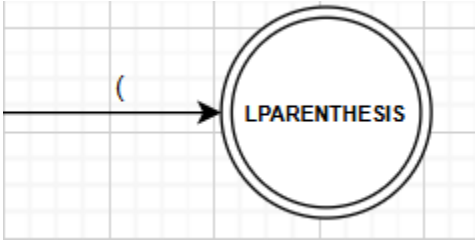
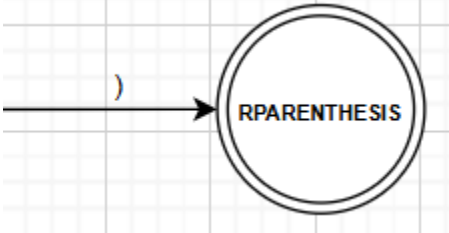
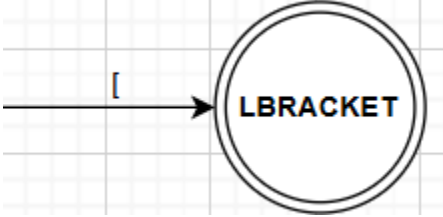
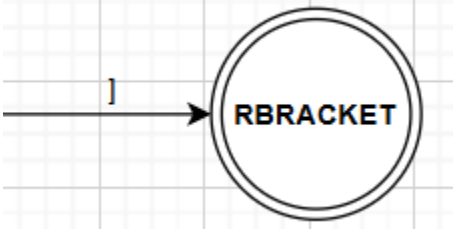
Diagram

Tipe	Diagram DFA	Penjelasan
KEYWORD	Sama seperti diagram DFA tipe IDENTIFIER (menggunakan lookup table)	KEYWORD terdiri atas sekumpulan kata-kata yang memiliki makna tertentu dan hanya mengandung karakter a-z. Tipe ini menggunakan diagram DFA IDENTIFIER kemudian dilakukan lookup pada tabel.
IDENTIFIER		Dari start state, ketika membaca karakter a-z maka akan berpindah ke state IDENTIFIER lalu state tidak berpindah selama karakter yang dibaca selanjutnya adalah a-z, 0-9.

ARITHMETIC_OPERATOR		Dari start state, ketika membaca karakter +, -, *, / maka berpindah ke state ARITHMETIC_OPERATOR. Sedangkan untuk div, mod menggunakan diagram DFA IDENTIFIER kemudian dilakukan lookup pada tabel.
RELATIONAL_OPERATOR		Dari start state, ketika membaca < maka akan berpindah ke state LT dan apabila selanjutnya membaca >, = maka akan berpindah ke state LT_X. Dari start state, ketika membaca > maka akan pindah ke state GT dan apabila selanjutnya membaca = maka akan berpindah ke state LT_X. Keempat state ini dilakukan pemetaan sehingga seluruhnya menunjukan tipe token RELATIONAL_OPERATOR
LOGICAL_OPERATOR	Sama seperti diagram DFA tipe IDENTIFIER (menggunakan lookup table)	LOGICAL_OPERATOR terdiri atas and, or, not. Tipe ini menggunakan diagram DFA IDENTIFIER kemudian dilakukan lookup pada tabel.
ASSIGN_OPERATOR		Dari start state, ketika membaca ":" maka akan berpindah ke state COLON dan ketika karakter selanjutnya adalah "=" maka akan berpindah ke state ASSIGN_OPERATOR.

NUMBER		Pada type number ada beberapa final state yang merupakan bagian dari type number. Final state tersebut adalah INT_LITRL, Neg_exp, Pos_exp. Jika state awal menerima sebuah number ia akan berpindah ke state INT_LITRL dan akan looping disana jika mendapat input angka terus menerus. Jika INT_LITRL menerima sebuah karakter e maka ia akan berpindah state ke Acc_Exp_lit yang merepresentasikan state saat mesin menerima notasi scientific. Acc_Exp_lit dapat berpindah ke 2 final state number yaitu Pos_exp dan Neg_exp. Ia akan berpindah ke Pos_exp jika state Acc_Exp_lit menerima sebuah tanda + atau angka 0-9. Namun jika ia menerima tanda - maka ia masuk ke state Neg_exp
CHAR_LITERAL DAN STRING_LITERAL		Untuk Char literal dari start state atau q0, ketika ia pertama kali membaca ' maka akan masuk ke q1, jika q1 menerima sebuah karakter setelahnya ia akan berpindah ke q2. Namun jika dia menerima ' state akan berpindah ke final state CHAR_LIT. State q2 juga akan berpindah ke final state CHAR_LIT jika menerima '. Ketika final state CHAR_LIT menerima sebuah ' maka state mesin akan berpindah ke state yang merepresentasikan ada double quote atau escape char (q5). Jika dari q2 atau CHAR_LIT atau q5 menerima char ia akan berpindah state

		ke q4 yang merepresentasikan kondisi ketika mesin menerima karakter setelah tanda kutip ganda atau setelah kutip tunggal, yang bukan kutip pembuka awal. Setelah itu jika q4 menerima sebuah quote ia akan masuk final state <code>STRING_LITERAL</code> yang merepresentasikan akhir dari string yang ditutup dengan <code>'</code>. Namun jika ternyata final state tersebut menerima <code>'</code> lagi yang berarti bukan akhir dari string ia akan berpindah ke state escape char (q5)
SEMICOLON	 <pre> graph LR Start(()) -- ";" --> SEMICOLON(((SEMICOLON))) </pre>	Dari start state jika dia membaca <code>";"</code> akan masuk ke state SEMICOLON
COMMA	 <pre> graph LR Start(()) -- "," --> COMMA(((COMMA))) </pre>	Dari start state, jika dibaca suatu karakter <code>","</code>, maka state berpindah ke final state COMMA
COLON	 <pre> graph LR Start(()) -- ":" --> COLON(((COLON))) </pre>	Dari start state, jika dibaca suatu karakter <code>":"</code>, maka state berpindah ke final state COLON

DOT		Dari start state, jika dibaca suatu karakter ".", maka state berpindah ke final state DOT
LPARENTHESIS		Dari start state, jika dibaca suatu karakter "(", maka state berpindah ke final state LPARENTHESIS
RPARENTHESIS		Dari start state, jika dibaca suatu karakter ")", maka state berpindah ke final state RPARENTHESIS
LBRACKET		Dari start state, jika dibaca suatu karakter "[", maka state berpindah ke final state LBRACKET
RBRACKET		Dari start state, jika dibaca suatu karakter "]", maka state berpindah ke final state RBRACKET

File Aturan DFA (.json)

```
{
  "start_state": "q0",
  "final_states": {
    "CHAR_LITERAL": { "type": "CHAR_LITERAL" },
    "STRING_LITERAL": { "type": "STRING_LITERAL" },
    "SEMICOLON": { "type": "SEMICOLON" },
    "COMMA": { "type": "COMMA" },
    "COLON": { "type": "COLON" },

```

```

"DOT":                { "type": "DOT" },
"LPARENTHESIS":      { "type": "LPARENTHESIS" },
"RPARENTHESIS":      { "type": "RPARENTHESIS" },
"IDENTIFIER":         { "type": "IDENTIFIER" },

"INT_LITRL":          { "type": "NUMBER" },
"Pos_exp":             { "type": "NUMBER" },
"Neg_exp":             { "type": "NUMBER" },

"ARITHMETIC_OPERATOR": { "type": "ARITHMETIC_OPERATOR" },
"STAR":                { "type": "ARITHMETIC_OPERATOR" },

"EQ":                 { "type": "RELATIONAL_OPERATOR" },
"LT":                 { "type": "RELATIONAL_OPERATOR" },
"LT_X":               { "type": "RELATIONAL_OPERATOR" },
"GT":                 { "type": "RELATIONAL_OPERATOR" },
"GT_E":               { "type": "RELATIONAL_OPERATOR" },

"LOGICAL_OPERATOR":   { "type": "LOGICAL_OPERATOR" },

"ASSIGN_OPERATOR":    { "type": "ASSIGN_OPERATOR" },

"LBACKET":            { "type": "LBACKET" },
"RBACKET":            { "type": "RBACKET" }

},

"LITERAL_STATES" : ["q1", "q2", "q4", "q5", "STRING_LITERAL",
"CHAR_LITERAL"],

"reserved_word": [
{ "lexeme": "program",  "token": "KEYWORD" },
{ "lexeme": "var",      "token": "KEYWORD" },
{ "lexeme": "begin",    "token": "KEYWORD" },
{ "lexeme": "end",      "token": "KEYWORD" },
{ "lexeme": "if",       "token": "KEYWORD" },
{ "lexeme": "then",     "token": "KEYWORD" },
{ "lexeme": "else",     "token": "KEYWORD" },
{ "lexeme": "while",    "token": "KEYWORD" },
{ "lexeme": "do",       "token": "KEYWORD" },
{ "lexeme": "for",      "token": "KEYWORD" },
{ "lexeme": "to",       "token": "KEYWORD" },
{ "lexeme": "downto",   "token": "KEYWORD" },
{ "lexeme": "integer",  "token": "KEYWORD" },
{ "lexeme": "real",     "token": "KEYWORD" },
{ "lexeme": "boolean",  "token": "KEYWORD" },
{ "lexeme": "char",     "token": "KEYWORD" },
{ "lexeme": "array",    "token": "KEYWORD" },
{ "lexeme": "of",       "token": "KEYWORD" },
{ "lexeme": "procedure", "token": "KEYWORD" },
{ "lexeme": "function", "token": "KEYWORD" },
{ "lexeme": "const",    "token": "KEYWORD" },
{ "lexeme": "type",     "token": "KEYWORD" },
{ "lexeme": "true",     "token": "KEYWORD" },
{ "lexeme": "false",    "token": "KEYWORD" },

```



```

    { "lexeme": "div",    "token": "ARITHMETIC_OPERATOR" },
    { "lexeme": "mod",    "token": "ARITHMETIC_OPERATOR" },

    { "lexeme": "and",    "token": "LOGICAL_OPERATOR" },
    { "lexeme": "or",     "token": "LOGICAL_OPERATOR" },
    { "lexeme": "not",    "token": "LOGICAL_OPERATOR" }
],

"transition_func" : [
    { "from": "q0", "input": "'", "to": "q1" },
    { "from": "q1", "input": "'", "to": "CHAR_LITERAL" },
    { "from": "q1", "input": "reg[^']", "to": "q2" },
    { "from": "q2", "input": "'", "to": "CHAR_LITERAL" },
    { "from": "q2", "input": "reg[^']", "to": "q4" },
    { "from": "q4", "input": "reg[^']", "to": "q4" },
    { "from": "q4", "input": "'", "to": "STRING_LITERAL" },
    { "from": "CHAR_LITERAL", "input": "'", "to": "q5" },
    { "from": "q5", "input": "'", "to": "STRING_LITERAL" },
    { "from": "q5", "input": "reg[^']", "to": "q4" },
    { "from": "STRING_LITERAL", "input": "'", "to": "q5" },

    { "from": "q0", "input": ";", "to": "SEMICOLON" },
    { "from": "q0", "input": ",", "to": "COMMA"},
    { "from": "q0", "input": ":", "to": "COLON"},
    { "from": "q0", "input": ".", "to": "DOT"},
    { "from": "q0", "input": "(", "to": "LPARENTHESIS"},
    { "from": "q0", "input": ")", "to": "RPARENTHESIS"},

    { "from": "q0", "input": "reg[A-Za-z]", "to": "IDENTIFIER"},
    { "from": "IDENTIFIER", "input": "reg[A-Za-z0-9]", "to": "IDENTIFIER" },

    { "from": "q0", "input": "reg[0-9]", "to": "INT_LITRL"},
    { "from": "INT_LITRL", "input": "reg[0-9]", "to": "INT_LITRL"},
    { "from": "INT_LITRL", "input": "E", "to": "Acc_Exp_lit"},
    { "from": "INT_LITRL", "input": "e", "to": "Acc_Exp_lit"},
    { "from": "INT_LITRL", "input": "reg[a-z^e]", "to": "INV_IDENTIFIER"},

    { "from": "Acc_Exp_lit", "input": "+", "to": "Pos_exp"},
    { "from": "Acc_Exp_lit", "input": "reg[0-9]", "to": "Pos_exp"},
    { "from": "Acc_Exp_lit", "input": "-", "to": "Neg_exp"},
    { "from": "Acc_Exp_lit", "input": "reg[A-Za-z]", "to": "INV_IDENTIFIER"},

    { "from": "Pos_exp", "input": "reg[0-9]", "to": "Pos_exp"},
    { "from": "Neg_exp", "input": "reg[0-9]", "to": "Neg_exp"},

    { "from": "INV_IDENTIFIER", "input": "reg[A-Za-z0-9]", "to":
"INV_IDENTIFIER"},

    { "from": "q0", "input": "+", "to": "ARITHMETIC_OPERATOR"},
    { "from": "q0", "input": "-", "to": "ARITHMETIC_OPERATOR"},
    { "from": "q0", "input": "/", "to": "ARITHMETIC_OPERATOR"},
    { "from": "q0", "input": "*", "to": "ARITHMETIC_OPERATOR"},

    { "from": "q0", "input": "=", "to": "EQ"},
    { "from": "q0", "input": "<", "to": "LT"},

```

```

{"from": "LT", "input": ">", "to": "LT_X"},
{"from": "LT", "input": "=", "to": "LT_X"},
{"from": "q0", "input": ">", "to": "GT"},
{"from": "GT", "input": "=", "to": "GT_E"},

{"from": "COLON", "input": "=", "to": "ASSIGN_OPERATOR"},

{"from": "q0", "input": "[", "to": "LBRACKET"},
{"from": "q0", "input": "]", "to": "RBRACKET"}

]
}

```

Implementasi Lexical Analyzer

```

import json
import sys

def match_pattern(pattern, char):
    if pattern == "reg[A-Za-z0-9_]":
        return char.isalnum() or char == "_"
    elif pattern == "reg[A-Za-z0-9]":
        return char.isalnum()
    elif pattern == "reg[A-Za-z]":
        return char.isalpha()
    elif pattern == "reg[0-9]":
        return char.isdigit()
    elif pattern == "reg[^]":
        return char != ""
    elif pattern == "reg[a-z^e]":
        return char.isalpha() and char != "e"
    else:
        return char == pattern

def get_next_state(current_state, char, dfa_json):
    for t in dfa_json["transition_func"]:
        if t["from"] == current_state and match_pattern(t["input"], char.lower()):
            return t["to"]
    return None

def check_keyword(stringValue, list_tok, dfa):
    for item in dfa["reserved_word"]:
        if item["lexeme"] == stringValue.lower():
            list_tok.append(f'{{item["token"]}}({stringValue})')
            return True
    return False

dfa = None
list_tokens = []

```

```

with open("src/DFA2.2.json", "r") as f:
    dfa = json.load(f)

try :
    file = sys.argv[1]
    with open(f"test/milestone-1/{file}.pas", "r") as f:
        pass
except FileNotFoundError:
    print("File tidak ditemukan. Pastikan file berada di folder 'test' dan berekstensi .pas")
    exit()

with open(f"test/milestone-1/{file}.pas", "r") as f:
    current_state = "q0"
    value = ""
    while True:
        #simpan pointer pos untuk jaga-jaga bila ada 1 token yang pembacaannya sudah
        #selesai yang ditandai dengan temp null
        pointerPos = f.tell()
        ch = f.read(1)

        temp = get_next_state(current_state, ch, dfa)

        if not ch: # EOF
            if current_state in dfa["final_states"].keys() and value:
                token_type = dfa["final_states"][current_state]["type"]
                list_tokens.append(f"{token_type}({value})")
                break

        if ch.isspace():
            #memastikan spasi dalam string literal tidak di anggap sebagai pemisah antar state
            if temp is not None:
                value += ch
                continue
            else:
                # jika spasi sebagai pemisah antar token
                if check_keyword(value, list_tokens, dfa):
                    value = ""
                    current_state = "q0"
                elif current_state in dfa["final_states"].keys() and value:
                    token_type = dfa["final_states"][current_state]["type"]
                    list_tokens.append(f"{token_type}({value})")
                elif value:
                    list_tokens.append(f"<ERROR>({value})")
                    value = ""
                    current_state = "q0"
                continue

```

#jika menemukan sebuah input char yang tidak menuju ke state manapun, maka akan dianggap pembacaan 1 token selesai

if temp is None:

if check_keyword(value, list_tokens, dfa):

f.seek(pointerPos)

elif current_state in dfa["final_states"].keys():

token_type = dfa["final_states"][current_state]["type"]

list_tokens.append(f"{token_type}({value})")

f.seek(pointerPos)

else:

list_tokens.append(f"<ERROR>({ch})")

#reset

current_state = "q0"

value = ""

else:

current_state = temp

value += ch

for token in list_tokens:

print(token)

PENGUJIAN

Kasus 1

Input file: test1.pas

```
program test1;

var
    x, y, total: integer;

begin
    x := 10;
    y := 5;
    total := x + y;
end.
```

Output

```
Masukkan nama file sumber (contoh: test1): test1
KEYWORD(program)
IDENTIFIER(test1)
SEMICOLON(;)
KEYWORD(var)
IDENTIFIER(x)
COMMA(,)
IDENTIFIER(y)
COMMA(,)
IDENTIFIER(total)
COLON(:)
KEYWORD(integer)
SEMICOLON(;)
KEYWORD(begin)
IDENTIFIER(x)
ASSIGN_OPERATOR(:=)
NUMBER(10)
SEMICOLON(;)
IDENTIFIER(y)
ASSIGN_OPERATOR(:=)
NUMBER(5)
SEMICOLON(;)
IDENTIFIER(total)
ASSIGN_OPERATOR(:=)
IDENTIFIER(x)
ARITHMETIC_OPERATOR(+)
IDENTIFIER(y)
SEMICOLON(;)
KEYWORD(end)
DOT(.)
```

Kasus 2

Input file: test2.pas

```
program test2;
```

```
var
    s: string;
    c: char;

begin
    s := 'Ini string';
    c := 'A';
end.
```

Output

```
Masukkan nama file sumber (contoh: test1): test2
KEYWORD(program)
IDENTIFIER(test2)
SEMICOLON(;)
KEYWORD(var)
IDENTIFIER(s)
COLON(:)
IDENTIFIER(string)
SEMICOLON(;)
IDENTIFIER(c)
COLON(:)
KEYWORD(char)
SEMICOLON(;)
KEYWORD(begin)
IDENTIFIER(s)
ASSIGN_OPERATOR(=)
STRING_LITERAL('Ini string')
SEMICOLON(;)
IDENTIFIER(c)
ASSIGN_OPERATOR(=)
CHAR_LITERAL('A')
SEMICOLON(;)
KEYWORD(end)
DOT(.)
```

Kasus 3

Input file: test3.pas

```
program test3;

var
    a, b: integer;

begin
    a := 7;
    b := 3;

    a := a + b;
    a := a - b;
    a := a * b;
    a := a / b;
    a := a div b;
    a := a mod b;
```

```
    if a = b then
        a := 0;

    if a <> b then
        a := 1;

    if a <= b then
        a := 2;

    if a >= b then
        a := 3;
end.
```

Output

```
Masukkan nama file sumber (contoh: test1): test3
KEYWORD(program)
IDENTIFIER(test3)
SEMICOLON(;)
KEYWORD(var)
IDENTIFIER(a)
IDENTIFIER(b)
COLON(:)
KEYWORD(integer)
SEMICOLON(;)
KEYWORD(begin)
IDENTIFIER(a)
ASSIGN_OPERATOR(:=)
NUMBER(7)
SEMICOLON(;)
IDENTIFIER(b)
ASSIGN_OPERATOR(:=)
NUMBER(3)
SEMICOLON(;)
IDENTIFIER(a)
ASSIGN_OPERATOR(:=)
IDENTIFIER(a)
ARITHMETIC_OPERATOR(+)
IDENTIFIER(b)
SEMICOLON(;)
IDENTIFIER(a)
ASSIGN_OPERATOR(:=)
IDENTIFIER(a)
ARITHMETIC_OPERATOR(-)
IDENTIFIER(b)
SEMICOLON(;)
IDENTIFIER(a)
ASSIGN_OPERATOR(:=)
IDENTIFIER(a)
ARITHMETIC_OPERATOR(*)
IDENTIFIER(b)
SEMICOLON(;)
IDENTIFIER(a)
ASSIGN_OPERATOR(:=)
```

```
IDENTIFIER(a)
ARITHMETIC_OPERATOR(/)
IDENTIFIER(b)
SEMICOLON(;)
IDENTIFIER(a)
ASSIGN_OPERATOR(:=)
IDENTIFIER(a)
ARITHMETIC_OPERATOR(div)
IDENTIFIER(b)
SEMICOLON(;)
IDENTIFIER(a)
ASSIGN_OPERATOR(:=)
IDENTIFIER(a)
ARITHMETIC_OPERATOR(mod)
IDENTIFIER(b)
SEMICOLON(;)
KEYWORD(if)
IDENTIFIER(a)
RELATIONAL_OPERATOR(=)
IDENTIFIER(b)
KEYWORD(then)
IDENTIFIER(a)
ASSIGN_OPERATOR(:=)
NUMBER(0)
SEMICOLON(;)
KEYWORD(if)
IDENTIFIER(a)
RELATIONAL_OPERATOR(<>)
IDENTIFIER(b)
KEYWORD(then)
IDENTIFIER(a)
ASSIGN_OPERATOR(:=)
NUMBER(1)
SEMICOLON(;)
KEYWORD(if)
IDENTIFIER(a)
RELATIONAL_OPERATOR(<=)
IDENTIFIER(b)
KEYWORD(then)
IDENTIFIER(a)
ASSIGN_OPERATOR(:=)
NUMBER(2)
SEMICOLON(;)
KEYWORD(if)
IDENTIFIER(a)
RELATIONAL_OPERATOR(>=)
IDENTIFIER(b)
KEYWORD(then)
IDENTIFIER(a)
ASSIGN_OPERATOR(:=)
NUMBER(3)
SEMICOLON(;)
KEYWORD(end)
DOT(.)
```


Kasus 4

Input file: test4.pas

```
program test4;

var
    x, y: integer;
    hasil: boolean;

begin
    x := 10;
    y := 5;
    hasil := (x > y) and not (x = y);

    if hasil or (x < y) then
        x := 1
    else
        x := 0;

    while x < 5 do
        x := x + 1;

    for x := 1 to 3 do
        y := y + x;
end.
```

Output

```
Masukkan nama file sumber (contoh: test1): test4
KEYWORD(program)
IDENTIFIER(test4)
SEMICOLON(;)
KEYWORD(var)
IDENTIFIER(x)
IDENTIFIER(y)
COLON(:)
KEYWORD(integer)
SEMICOLON(;)
IDENTIFIER(hasil)
COLON(:)
KEYWORD(boolean)
SEMICOLON(;)
KEYWORD(begin)
IDENTIFIER(x)
ASSIGN_OPERATOR(:=)
NUMBER(10)
SEMICOLON(;)
IDENTIFIER(y)
ASSIGN_OPERATOR(:=)
NUMBER(5)
SEMICOLON(;)
IDENTIFIER(hasil)
ASSIGN_OPERATOR(:=)
LPARENTHESIS( ( )
IDENTIFIER(x)
```

```
RELATIONAL_OPERATOR(>)
IDENTIFIER(y)
RPARENTHESIS() )
LOGICAL_OPERATOR(and)
LOGICAL_OPERATOR(not)
LPARENTHESIS( )
IDENTIFIER(x)
RELATIONAL_OPERATOR(=)
IDENTIFIER(y)
RPARENTHESIS() )
SEMICOLON(;)
KEYWORD(if)
IDENTIFIER(hasil)
LOGICAL_OPERATOR(or)
LPARENTHESIS( )
IDENTIFIER(x)
RELATIONAL_OPERATOR(<)
IDENTIFIER(y)
RPARENTHESIS() )
KEYWORD(then)
IDENTIFIER(x)
ASSIGN_OPERATOR(:=)
NUMBER(1)
KEYWORD(else)
IDENTIFIER(x)
ASSIGN_OPERATOR(:=)
NUMBER(0)
SEMICOLON(;)
KEYWORD(while)
IDENTIFIER(x)
RELATIONAL_OPERATOR(<)
NUMBER(5)
KEYWORD(do)
IDENTIFIER(x)
ASSIGN_OPERATOR(:=)
IDENTIFIER(x)
ARITHMETIC_OPERATOR(+)
NUMBER(1)
SEMICOLON(;)
KEYWORD(for)
IDENTIFIER(x)
ASSIGN_OPERATOR(:=)
NUMBER(1)
KEYWORD(to)
NUMBER(3)
KEYWORD(do)
IDENTIFIER(y)
ASSIGN_OPERATOR(:=)
IDENTIFIER(y)
ARITHMETIC_OPERATOR(+)
IDENTIFIER(x)
SEMICOLON(;)
KEYWORD(end)
DOT(. )
```

Kasus 5

Input file: test5.pas

```
program test5;

var
    angka: array[1..5] of integer;
    i: integer;

begin
    for i := 1 to 5 do
        angka[i] := i;

        angka[3] := 3.1415E+2;
    end.
```

Output

```
Masukkan nama file sumber (contoh: test1): test5
KEYWORD(program)
IDENTIFIER(test5)
SEMICOLON(;)
KEYWORD(var)
IDENTIFIER(angka)
COLON(:)
KEYWORD(array)
LBRACKET([)
NUMBER(1)
DOT(.)
DOT(.)
NUMBER(5)
RBRACKET(])
KEYWORD(of)
KEYWORD(integer)
SEMICOLON(;)
IDENTIFIER(i)
COLON(:)
KEYWORD(integer)
SEMICOLON(;)
KEYWORD(begin)
KEYWORD(for)
IDENTIFIER(i)
ASSIGN_OPERATOR(:=)
NUMBER(1)
KEYWORD(to)
NUMBER(5)
KEYWORD(do)
IDENTIFIER(angka)
LBRACKET([)
IDENTIFIER(i)
RBRACKET(])
ASSIGN_OPERATOR(:=)
IDENTIFIER(i)
SEMICOLON(;)
IDENTIFIER(angka)
```

```
LBRACKET ( [ )
NUMBER ( 3 )
RBRACKET ( ] )
ASSIGN_OPERATOR ( := )
NUMBER ( 3 )
DOT ( . )
NUMBER ( 1415E+2 )
SEMICOLON ( ; )
KEYWORD ( end )
DOT ( . )
```

KESIMPULAN DAN SARAN

Pada Milestone 1 ini, kami telah merancang diagram transisi DFA serta mengimplementasikan lexical analyzer (lexer) untuk bahasa Pascal-S. Lexer ini dapat mengidentifikasi token-token pada kode sumber (.pas) sesuai dengan aturan yang berlaku, seperti KEYWORD, IDENTIFIER, NUMBER dan lain sebagainya.

LAMPIRAN

- <https://github.com/jhotlann/LPH-Tubes-IF2224>
- [Link workspace diagram](#)
- Pembagian Tugas:

No.	NIM	Persentase Kontribusi
1.	13523023	25%
2.	13523025	25%
3.	13523051	25%
4.	13523111	25%

REFERENSI

Aho, Alfred V. *Compilers: Principles, Techniques, & Tools*. Pearson/Addison Wesley, 2007,
[https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf). Accessed 16 October 2025.

“Compiler Design - Lexical Analysis.” *Tutorials Point*,

https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm.

Accessed 16 October 2025.

Hopcroft, John E., et al. *Introduction to Automata Theory, Languages, and Computation*. 3rd ed.,
Pearson/Addison Wesley, 2007,

https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613_Introduction-to-Automata-Theory,-Languages,-and-Computation-Edition-3.pdf.

Accessed 16 October 2025.

Wirth, Niklaus. *PASCAL-S. A Subset and its Implementation*.

<http://pascal.hansotten.com/uploads/pascals/PASCAL-S%20A%20subset%20and%20its%20Implementation%20012.pdf>. Accessed 16 October 2025.